Lab-2:Model Engineering
Group-13

Amirali Eghdami Malati- s328630,
Hicham El kettani- s344121, Jingbin Hu- s336634

1 Task 1: Frequency-based baseline

Q: How many unique API calls does the training set contain? How many the test set?
A: The training set contains **258** unique API calls, and the test set contains **232** unique API calls.

Q: Are there any API calls that appear only in the test set (but not in the training set)?If yes, how many? And which one are they?
A: Yes, there are **3** API calls that appear exclusively in the test set, which are `WSASocketA`, `ControlService`, and `NtDeleteKey`.

Q: Can you use the test vocabulary to build the new test dataframe? If not, how do you handle API calls in the test set that do not exist in the training vocabulary?
A: No, we cannot use the test vocabulary to build the test dataframe because the training and test sets must maintain the exact same feature dimensions in machine learning. While the training vocabulary defines a **258-dimensional** feature space, the test set only contains 232 unique APIs, including **3 out-of-vocabulary (OOV)** APIs that do not exist in the training set. Therefore, to ensure dimensional consistency, we build the test dataframe strictly using the **training vocabulary** and completely **ignore (drop)** any unseen API calls during the frequency counting process.

Q: One issue of this frequency-based approach is that it creates sparse vectors (i.e., vectors with many zeros per row): how many non-zero elements per row do you have on average in the training set? How many in the test set? What is the ratio with respect to the number of elements per row?

Table 1: Sparsity and Non-Zero Element Analysis

Dataset Partition	Avg. Non-Zero Elements	Non-Zero Ratio	Sparsity (Zero Ratio)
Training Set	21.95 / 258	8.51%	91.49%
Test Set	24.28 / 258	9.41%	90.59%

These empirical results confirm that the frequency-based feature matrices are **highly sparse** (with a sparsity/zero-ratio exceeding **90%** for both partitions), as each malware sequence triggers only a small subset of the global API vocabulary.

Q: The original API sequences were ordered. Is it still the case now (i.e., in the frequency-based dataframe, do you still know which API came first)? Why?

A: No, it is no longer the case. In the frequency-based dataframe, the original temporal order and execution sequence of the API calls are **completely lost**. This is because the frequency-based approach compresses the sequence into a "Bag-of-Words" representation, where we only count the total occurrences of each API call across the entire process execution. The dataframe columns represent fixed, static vocabulary dimensions rather than consecutive time steps, making it impossible to reconstruct which API came first or map any contextual transitions between consecutive calls.

Q: Report how you chose the hyperparameters of your classifier, and the final performance on the test set.

A: Before conducting the hyperparameter search, the **Random Forest Classifier** was selected as our baseline specifically due to the intrinsic characteristics of the vectorized datasets:

- **High Sparsity and High Dimensionality:** Our feature engineering stage yielded a 258-dimensional feature space where over **90%** of the matrix elements are zeros. Tree-based ensemble models like

Random Forest handle sparse tabular representations exceptionally well since they partition data using simple, robust deterministic splits (e.g., whether an API count is greater than zero) rather than continuous multi-dimensional distance or matrix weight multiplications.

- **Extreme Class Imbalance:** With malware making up over **96%** of the training set, a native model would easily overfit to the majority class. Random Forest allows the mitigation of this via the `class_weight="balanced"` strategy, which heavily penalizes mistakes on the rare goodware samples.

To systematically determine the optimal architecture within this sparse space, we executed a 3-fold cross-validation grid search (`GridSearchCV`) on the training set, optimizing for the Macro F_1-score. The hyperparameter search boundaries and final selections are shown in Table 2, and the final performance metrics evaluated on the test set are summarized in Table 3.

Table 2: Random Forest Hyperparameter Search and Selection

Hyperparameter	**Search Space**	**Optimal Selection**
Splitting Criterion	[`gini`, `entropy`]	`gini`
Maximum Depth (`max_depth`)	[10, 20, `None`]	`None`
Number of Trees (`n_estimators`)	[50, 100, 200]	50

Table 3: Final Classifier Performance on Test Set

Class	**Precision**	**Recall**	**F_1-Score**	**Support**
Goodware (0)	0.8318	0.3663	0.5086	243
Malware (1)	0.9759	0.9971	0.9864	6262
Accuracy			0.9736	6505
Macro Avg	0.9039	0.6817	0.7475	6505
Weighted Avg	0.9705	0.9736	0.9686	6505

Q: Is the final performance good–even ignoring the order of API calls and handling very sparse vectors?

A: Yes, the overall performance is robust, but it reveals a critical flaw regarding the minority class due to the loss of sequential context:

- **High Malware Detection Rate:** The baseline achieves an overall Accuracy of **97.36%** and a Malware F_1-score of **0.9864**. This proves that despite **90%+ sparsity** and zero temporal order, the sheer frequency of certain malicious APIs remains a strong, discriminative indicator of compromise.
- **Low Goodware Recall (The Cost of Dropping Order):** However, the model yields a low Recall of **36.63%** for Goodware (F_1-score of **0.5086**), resulting in 154 false positives. This demonstrates the limitation of a bag-of-words approach: legitimate programs often call the exact same APIs as malware, but they execute them in a completely different, benign chronological order.

2 Task 2: Feed Forward Neural Network (FFNN)

Q: Do you have the same number of API calls for each sequence? If not, is the distribution of the number of API calls per sequence in the training set the same as the test one?

A: No, we do not have the same number of API calls for each sequence. The execution traces vary significantly in length depending on the software's dynamic behavior.
Furthermore, the **distribution** of the sequence lengths in the training set is **not the same** as the test set. A distinct distributional mismatch (covariate shift) is observed between the two partitions, as summarized in Table 4 and visualized in Figure 1.
As illustrated by the empirical metrics and the density histogram, there is a clear rigid displacement between the two datasets: the training sequences are strictly bounded between 60 and 90, whereas the test sequences are shifted to the right, ranging from 70 to 100.

Table 4: Descriptive Statistics of API Sequence Lengths

Dataset	Mean Length	Std. Dev.	Min	Median (50%)	Max
Training Set	75.03	8.95	60.0	75.0	90.0
Test Set	86.33	9.11	70.0	87.0	100.0

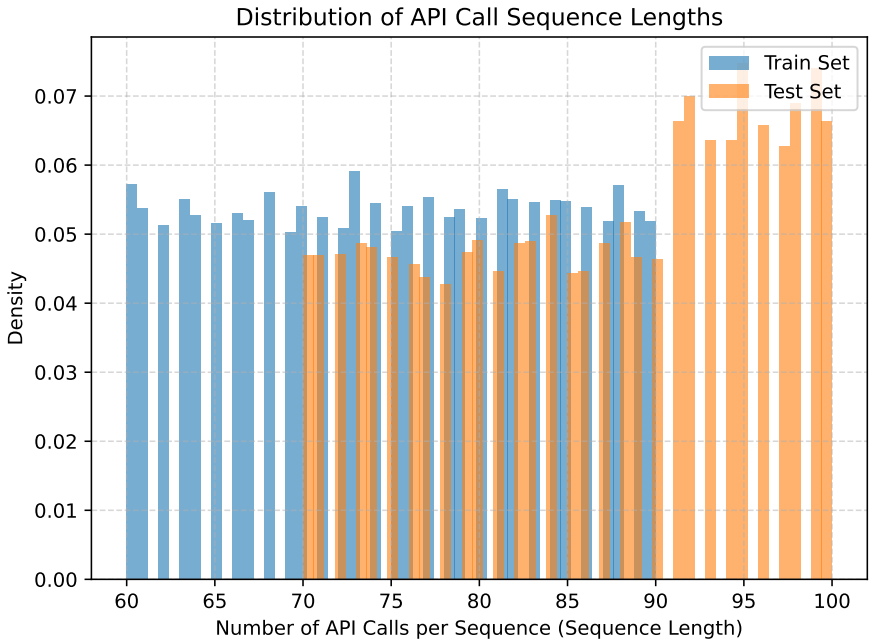

Figure 1: Comparison of API Call Sequence Length Distributions

Q: The first neural approach we learned involves FFNN. Can a FFNN handle a variable number of elements? If not, why?

A: No, a standard Feed-Forward Neural Network (FFNN) cannot natively handle a variable number of input elements.

The structural and mathematical reasons for this architectural limitation include:

- **Fixed Input Layer Dimension:** An FFNN requires a strictly predefined, static number of input neurons. Once the network topology is defined, the size of the input layer cannot dynamically expand or shrink to accommodate varying input sequence lengths.
- **Mathematical Dimension Mismatch:** The core forward-propagation step in a dense layer relies on linear algebra matrix multiplication: $Y = \sigma(W \cdot X + b)$, where W represents the static weight matrix and X represents the input feature vector. If the dimension of X varies between samples, the matrix multiplication becomes algebraically impossible due to a shape mismatch.
- **Lack of Temporal Weight Sharing:** Unlike Recurrent Neural Networks (RNNs) that process variables sequentially step-by-step using a temporal loop, an FFNN expects all features to be presented simultaneously in a single forward pass, lacking any mechanism to process elements one by one over arbitrary sequence lengths.

Q: How to estimate a *fixed-size* candidate?

A: To determine a fixed-size sequence length (L) suitable for a Feed-Forward Neural Network (FFNN), we considered two primary standard methodologies:

- **Lossless Maximum Length (Max Length):** Selecting the absolute maximum length present in the dataset to ensure zero information loss, padding shorter sequences with zeros to match this boundary.
- **Percentile-Based Truncation (Percentile):** Selecting a high threshold (e.g., 95th percentile) to truncate rare, exceptionally long outlier sequences, preventing the input matrix from becoming excessively bloated and sparse.

Based on the empirical characteristics of our data, we chose the **Lossless Maximum Length** method, setting our fixed-size candidate to **90**. Our exploratory data analysis showed that the training sequences are tightly distributed between 60 and 90 with a mean of 75.03. Because there is no heavy-tailed distribution or extreme outlier spikes, choosing the maximum value of 90 allows us to preserve 100% of the behavioral API information without incurring prohibitive padding overhead.

Q: What partition do you use to estimate it?

Crucially, this estimation was conducted strictly using the **training set partition**. Utilizing the test set to determine preprocessing configurations would introduce **data leakage** (lookahead bias), violating the constraint that the test set must simulate completely unseen deployment data.

Q: Given the estimate of the previous point, what technique could you use to obtain the same number of API calls per sequence?

A: To enforce the uniform length of 90 API calls per sequence required by the static architecture of the FFNN, we can use a combination of **Zero-Padding** and **Truncation**:

- **Zero-Padding (for shorter sequences):** For sequences with fewer than 90 API calls (the training samples range from 60 to 89), a neutral padding token (`0` or `[PAD]`) is appended to the end of the sequence until it reaches exactly 90 elements.
- **Truncation (for longer sequences):** For sequences exceeding 90 API calls (which occurs only in the test set, up to 100), the trailing execution calls are discarded to strictly fit the 90-element boundary.

Q: Suppose that at test time you have more API calls than the fixed-size you estimated. What do you do with the API calls exceeding such fixed-size?

A: The API calls exceeding the fixed size are handled via **Truncation**. Specifically, any trailing API calls that occur beyond the predetermined 90-element threshold are simply **discarded (dropped)**.

Q: Use a FFNN in both cases. Report how you selected the hyper-parameters of your final model, and justify your choices.

Hyper-parameter Selection: The final configuration was determined via a **16-combination Full Grid Search** implemented inside an isolated validation framework:

- **Grid Permutations:** Hidden units $H \in \{16, 32\}$, embedding sizes $\text{Dim} \in \{8, 16, 32\}$, and mini-batch sizes $\text{Batch} \in \{32, 64\}$.
- **Fixed Controls:** Learning rate locked at $\eta = 0.001$ (Adam optimizer) paired with a post-embedding Dropout rate of 0.5.
- **Epoch Regulation:** Upper-bounded at 30 epochs, dynamically managed by an in-memory Early Stopping mechanism (patience = 5 epochs monitored on validation loss).

Architectural Justifications: The core rationales behind these structural design choices include:

- **Information Bottleneck Enforcement ($H16$ / $H32$):** Deep multi-layer networks overfit by memorizing absolute token positions. Since malicious call chains are structurally low-dimensional, a tight single-layer bottleneck forces the network to filter out trace noise and extract generalizing malware abstractions.
- **Causal Sequence Retention via Flattening:** Mean-pooling averages across time to compress a $(\text{Batch}, 90, \text{Dim})$ tensor into $(\text{Batch}, \text{Dim})$. However, because vector addition is commutative ($A + B = B + A$), pooling washes out execution order. Spatial flattening preserves absolute positional indices, keeping the chronological execution sequence intact.
- **Sparse Embedding Compression ($\text{Dim} \in \{8, 16, 32\}$):** Given a highly constrained vocabulary ($\text{Vocab} \approx 259$), oversized dimensions (e.g., 64 or 128) disperse discrete tokens across excessively wide continuous spaces, raising generalization risks. Highly compressed dimensions map behavioral semantics cleanly without fitting vocabulary noise.

Q: Can you obtain the same results for the two alternatives (sequential identifiers and learnable embeddings)? If not, why?

A: Comparison of Input Representations: No, the two alternatives yield drastically different results.

Architecture Setup	TEST Macro F1	TEST ROC-AUC	Benign Recall	TN / FP
Case 1: Seq ID (Best: H32 | B64)	49.05%	65.89%	0.00	0 / 243
Case 2: Emb (Best: H32 | B64 | Dim 32)	56.07%	83.41%	0.08	19 / 224

Table 5: Key Performance Indicators: Sequential IDs vs. Learnable Embeddings

While learnable embeddings successfully unlock the network's representation capacity compared to the catastrophic failure of sequential identifiers, the absolute performance on the minority class remains severely bottlenecked in both cases due to extreme class imbalance.

- **Majority Class Prediction Collapse (Sequential IDs):** The `TN / FP` column reveals a catastrophic failure mode for the sequential ID approach. Generating exactly `0` True Negatives means the network completely failed to find a classification boundary. By mapping categorical API strings to arbitrary sequential integers, we introduce a false ordinal magnitude (e.g., implying API #2 is mathematically closer to #3 than to #90). The network defaults to predicting the majority class (Malware) for every sample, stalling the Macro F1 at 49.05%.
- **Semantic Vector Space Extraction (Learnable Embeddings):** Conversely, Learnable Embeddings project discrete API calls into a continuous, high-dimensional vector space, allowing the model to cluster semantically related behaviors. This structural advantage is proven by the significant jump in the TEST ROC-AUC (from $\sim 65\%$ to $\sim 83\%$), indicating that the embeddings successfully improved the probabilistic ranking and separation of the two classes in the latent space.
- **The "Recall < 0.1" Bottleneck (Imbalance & Architectural Limits):** Despite the superior representation of embeddings, the model still heavily underperforms on the minority class, peaking at a Benign Recall of just 0.08 (19 True Negatives out of 243). This highlights two critical realities: first, the extreme dataset imbalance (6262 Malware vs. 243 Benign) overwhelms the cross-entropy loss, skewing the final decision boundary toward Malware regardless of the embeddings. Second, a simple FFNN inherently ignores the sequential order of API calls—a flaw that embeddings alone cannot fix, paving the way for inherently sequential models like RNNs.

3 Task 3: Recursive Neural Network (RNN)

Q: With RNNs, do you still have to pad your data? If yes, how?

A: Yes, padding is strictly required for parallelized mini-batch training on GPUs, though its algorithmic side effects must be neutralized.

The core reasons and engineering mechanics include:

- **Mini-Batch Tensor Constraints:** While RNN cells process arbitrary lengths step-by-step, GPU hardware requires uniform dimensions. Sequences must be post-padded to a fixed maximum length to form a valid 3D tensor (Batch Size, Sequence Length, Feature Dim).
- **Hidden State Contamination:** Padding introduces meaningless trailing tokens. Left unmanaged, the RNN continues matrix operations over these empty steps, actively diluting and distorting the valid behavioral features stored in the historical hidden state.
- **Neutralization via Packing:** To eliminate weight bias, true unpadded sequence lengths are tracked and passed to `pack_padded_sequence`. This instructs the internal recurrence loop to bypass padding tokens, freezing the hidden state immediately after each sample's last valid API call.

Q: Do you have to truncate the testing sequences? Justify your answer with your understanding of why it is/it is not the case.

A: Yes, testing sequences must be truncated (and padded) to match the identical maximum length threshold established during training (e.g., 90 timesteps), despite the theoretical capacity of RNNs to process arbitrary lengths.

The empirical and mathematical justifications for this enforcement include:

- **Parallelized Evaluation Efficiency:** Although single-instance inference (Batch Size = 1) mathematically bypasses shape constraints, evaluating large test partitions sequence-by-sequence defeats

hardware parallelism. To leverage GPU accelerated batch-inference, the entire testing matrix must be structured into a uniform tensor, necessitating standardized padding and truncation.

- **Prevention of Out-of-Distribution (OOD) Degradation:** An RNN's weight matrices are optimized to capture behavior dynamics constrained within a specific historical horizon (90 steps). Feeding raw, excessively long test sequences introduces a distribution shift where compounding recurrent transitions cause hidden state saturation or activation drift, leading to erratic classification failures.

Q: Is the RNN padding – if any – more memory efficient with respect to the FFNN's one? Why?
A: Padding Memory Efficiency: Yes, the padding mechanism in our RNN implementation is significantly more memory efficient than that of the FFNN architecture.

- **FFNN Static Allocation:** In a standard FFNN, the input tensor must be flattened to a fixed size. Consequently, the zero-padding tokens are treated as active features that propagate through all dense layers, forcing the GPU to statically allocate memory to store intermediate activations and backward gradients for these useless zeros.
- **Dynamic Packing Mechanism:** In contrast, our RNN/LSTM pipeline utilizes the `pack_padded_sequence` mechanism, which dynamically strips away the zero-padding tokens from the computational graph prior to recurrent execution.
- **Computation Truncation:** The RNN internal cell completely ceases internal computation for a specific sample sequence as soon as its true sequence boundary is reached.
- **Resource Optimization:** Therefore, the dynamic runtime memory specifically for tracking hidden states, cell states, and backpropagation trajectories is exclusively allocated for actual valid tokens, achieving optimal memory efficiency.

Q: Start with a simple one-directional RNN. Is your network fast as the FFNN? If not, where do you think that time-overhead comes from?
A: Network Speed and Time-Overhead: No, the RNN is significantly slower than the FFNN due to sequential dependencies that prevent parallel computation and expand the computational graph along the time dimension.

- **Sequential Dependency:** Unlike the FFNN's single-shot parallel matrix multiplication, the RNN must compute hidden states step-by-step ($h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b)$), forcing a sequential loop that severely underutilizes GPU parallelism.
- **Memory Bottleneck:** The recurrent cell must repeatedly fetch weight matrices (W_{xh}, W_{hh}) from global memory at each individual time step, making the execution heavily memory-bandwidth bound.
- **BPTT Graph Depth:** Backpropagation Through Time (`BPTT`) unrolls the network along the sequence length T. Gradients must propagate through long temporal chains via consecutive matrix multiplications, accumulating massive computational latency.
- **Activation Caching:** To compute backward gradients, the GPU cannot discard intermediate states and must cache all hidden states h_t across all T time steps, introducing substantial memory allocation and management overhead.

Q: Train and test three variations of networks
A: Model Overview and Hyperparameter Setup: To evaluate sequential modeling for malware detection, we trained and evaluated the following four distinct recurrent architectures:

1. **Unidirectional RNN** (`SimpleRNNClassifier`)
2. **Bidirectional RNN** (`BiRNNClassifier`)
3. **Unidirectional LSTM** (`UnidirectionalLSTMClassifier`)
4. **Bidirectional LSTM** (`BidirectionalLSTMClassifier`)

Architecture-Specific Learning Rates:

Common Configuration Parameter	Shared Value / Setting
Vocabulary Size (`vocab_size`)	Total unique tokens in `train_vocab`
Embedding Dim (`embedding_dim`)	32
Hidden Size (`hidden_size`)	32
Number of Layers (`num_layers`)	1
Dropout Rate (`dropout_p`)	0.5
Batch Size	32
Max Epochs (`num_epochs`)	30
Early Stopping Patience	5
Loss Function (`criterion`)	`BCEWithLogitsLoss`
Optimizer Type	Adam

Table 6: Shared Global Configuration Settings

- **Vanilla RNN Models (Uni / Bi):** Learning Rate = 0.0005
- **LSTM Models (Uni / Bi):** Learning Rate = 0.001

Rationale for Decoupled Learning Rates:

- **RNN Stability Constraints (**0.0005**):** Vanilla recurrent cells lack specialized gating controls, rendering backpropagation through time (`BPTT`) highly volatile and prone to gradient explosions. Utilizing a more conservative, smaller learning rate stabilizes optimization down its highly non-linear and rugged loss surface.
- **LSTM Gating Robustness (**0.001**):** LSTMs track dependencies via an internal cell state regulated by explicit input, forget, and output gates. This inherent architectural protection naturally counteracts vanishing and exploding gradients, allowing the optimization engine to safely deploy a standard, larger learning rate to accelerate convergence.

Q: Is the RNNs training as stable as the FFNN's one?
A: Training Stability: No, RNN training is significantly less stable than FFNN training, primarily due to the severe vanishing and exploding gradient problems inherent in backpropagation through time.

- **Vanishing and Exploding Gradients:** As gradients propagate backward through T time steps, they repeatedly multiply the same recurrent weight matrix W_{hh}. This causes the gradient to grow or decay exponentially ($\|W_{hh}\|^T$), leading to numerical instability or stalled learning.
- **Rugged Loss Landscape:** Temporal dependencies create a highly non-linear, complex loss surface characterized by sharp cliffs and deep valleys. This makes optimization highly erratic, causing sudden jumps in loss compared to the smoother surfaces of FFNNs.
- **Sensitivity to Initialization:** RNNs are extremely sensitive to initial weight scales and learning rates. Incorrect initialization quickly triggers divergence, requiring specialized techniques like orthogonal initialization or identity mapping to remain viable.

Q: How does your model's performance compare to the simple frequency baseline, given that you now account for the sequence of API calls and use a significantly more complex network?

Model	Accuracy	Benign Precision	Benign Recall	Benign F1	ROC-AUC
Frequency Baseline	97.36%	**83.18%**	36.63%	50.86%	N/A
Unidirectional RNN	96.77%	67.01%	26.75%	38.24%	0.8538
Bidirectional RNN	96.43%	60.38%	13.17%	21.62%	0.8463
Unidirectional LSTM	**97.36%**	75.18%	**43.62%**	**55.21%**	**0.9494**
Bidirectional LSTM	97.08%	67.55%	41.98%	51.78%	0.9346

Table 7: Performance Comparison: RNN Variations vs. Frequency Baseline

4 Task 4: Graph Neural Network (GNN)

Q: Do you still have to pad your data? If yes, how?
A: No, padding is not required. PyTorch Geometric (PyG) batches graphs by merging them into a single giant disjoint graph (disjoint union) via the following mechanism:

- **Node Features:** Feature matrices $X_i \in \mathbb{R}^{N_i \times D}$ are vertically concatenated into a unified matrix $X \in \mathbb{R}^{(\sum N_i) \times D}$, since the feature dimension D is invariant.
- **Edge Indices:** Sparse COO matrices (`edge_index` of shape $2 \times |E|$) are horizontally concatenated. Node indices of subsequent graphs are automatically shifted by $\sum_{j<i} N_j$ to prevent cross-graph connections.
- **Batch Vector:** An automatically tracking `batch` vector $\mathbf{b} \in \{0, \ldots, B-1\}^{\sum N_i}$ maps each node to its respective graph, enabling global pooling layers to aggregate individual graph-level representations.

Q: Do you have to truncate the testing sequences? Justify your answer with your understanding of why it is/it is not the case.
A: No, truncation is not required for Graph Neural Networks (GNNs). The model's architecture natively handles variable-sized inputs through the following principles:

- **Size-Invariant Message Passing:** GNNs use shared weight matrices across all nodes and edges. Because the message-passing operation is defined by the graph's connectivity rather than a fixed input dimension, the model can process any number of API calls (nodes) and transitions (edges) dynamically.
- **Global Pooling:** To interface with a fixed-size output layer, GNNs employ global pooling (e.g., `global_mean_pool`). This step aggregates an arbitrary number of node embeddings into a single vector of constant length, making input-level truncation mathematically unnecessary.
- **Memory Efficiency:** In this specific dataset, API sequences are relatively short (up to 100 non-repeated calls). This size is well within the memory capacity of modern GPUs, allowing the model to preserve the full structural information of the software's behavior during testing.

Q: What is the advantage of modeling your problem with a GNN with respect to an RNN in this scenario? However, what do you lose?
A: In API-based malware classification, the architectural trade-offs between a GNN and an RNN are:
What You Gain (Advantages over RNN):

- **Obfuscation Resilience:** Malware often injects junk API calls to disrupt sequential patterns. An RNN easily loses track over long, diluted sequences. A GNN preserves the core exploit signature as localized graph topologies, making it immune to linear padding.
- **Non-Linear Flow Modeling:** Software execution naturally contains loops and branches. While an RNN forces this into an artificial, flat 1D timeline, a GNN maps APIs as nodes and transitions as directed edges, natively reflecting the true control flow graph.

What You Lose (The Trade-offs):

- **Exact Chronological Path:** Compressing a sequence into unique nodes flattens the exact timeline. For example, a cyclic pattern like $A \to B \to A \to B$ collapses into two bidirectionally connected nodes, discarding the precise step-by-step trajectory.
- **Frequency and Scale Sensitivity:** Standard graph adjacency matrices are binary (tracking if a transition happened, not how often). Structurally, calling an API once can look identical to executing it 10,000 times in a loop, whereas an RNN naturally accumulates this volume via continuous state updates.

Q: Finally train and tune variations of GNN considering different message and aggregation functions and architectures: Simple GCN, GraphSAGE, and GAT.
A: To evaluate the role of topological message passing and aggregation mechanisms for API-based malware classification, a structured grid search was executed ($D_{emb} = 32$, $D_{hidden} = 64$, $D_{layer} = 2$). The architectural tuning variations are organized as follows:
Key Methodological Formulations:

Table 8: Hyperparameter Search Space Across GNN Architectures

Architecture	Message Passing	Local (Neighbor) Aggregation	Global Graph Readout	Learning Rate (lr)
Simple GCN	Isotropic (Degree-Norm)	Sum (Fixed)	`mean`, `max`, `sum`	0.010
GraphSAGE	Feature Preservation	`mean`, `max`, `sum`	`mean`, `max`, `sum`	0.005
GAT	Anisotropic (Attention)	Multi-Head ($H \in \{1, 4, 8\}$)	`mean`, `max`, `sum`	0.002

- **Simple GCN (Isotropic Base):** Uses a static structural scaling factor $\frac{1}{\sqrt{\tilde{\deg}(i) \cdot \tilde{\deg}(j)}}$. Tuning is applied strictly to global readouts to benchmark how baseline topology interacts with macro-level pooling.
- **GraphSAGE (Inductive Base):** Evaluates the raw impact of neighborhood feature scaling. The 2D grid optimizes the interaction between localized feature distillation (*Local Aggregation*) and graph flattening (*Global Readout*).
- **GAT (Dynamic Attention Base):** Formulates message weights via data-dependent attention coefficients α_{ij}. The grid explores the expressive capacity of parallel attention streams (H) against macro graph summaries.

Q: How does each model perform with respect to the previous architectures? Can you beat the baseline?

Model / Pipeline Stage	Acc	B-Prec	B-Recall	B-F1	Macro F1
Frequency Baseline (Task 1)	97.36%	83.18%	36.63%	50.86%	74.75%
1. GCN Models (3 variants)					
— GCN-E32-Amax	96.26%	0.00%	0.00%	0.00%	49.05%
— GCN-E32-Amean	96.30%	56.25%	3.70%	6.95%	52.53%
— GCN-E32-Asum	96.37%	60.00%	8.64%	15.11%	56.63%
2. GraphSAGE Models (9 variants)					
— SAGE-L_mean-G_mean	96.69%	71.88%	18.93%	29.97%	64.14%
— SAGE-L_mean-G_max	96.77%	**86.67%**	16.05%	27.08%	62.72%
— SAGE-L_mean-G_sum	97.22%	71.83%	41.98%	52.99%	75.78%
— SAGE-L_max-G_mean	**97.54%**	81.68%	44.03%	57.22%	77.98%
— SAGE-L_max-G_max	97.46%	72.94%	51.03%	60.05%	79.37%
— **SAGE-L_max-G_sum (Best)**	97.43%	70.88%	**53.09%**	**60.71%**	**79.69%**
— SAGE-L_sum-G_mean	97.25%	79.09%	35.80%	49.29%	73.94%
— SAGE-L_sum-G_max	97.52%	80.15%	44.86%	57.52%	78.12%
— SAGE-L_sum-G_sum	97.42%	76.60%	44.44%	56.25%	77.46%
3. GAT Models (9 variants)					
— GAT-H_1-G_mean	96.51%	69.05%	11.93%	20.35%	59.28%
— GAT-H_1-G_max	96.26%	0.00%	0.00%	0.00%	49.05%
— GAT-H_1-G_sum	96.74%	68.24%	23.87%	35.37%	66.85%
— GAT-H_4-G_mean	96.54%	78.12%	10.29%	18.18%	58.21%
— GAT-H_4-G_max	96.53%	75.76%	10.29%	18.12%	58.17%
— **GAT-H_4-G_sum (Best GAT)**	96.99%	85.07%	23.46%	36.77%	67.62%
— GAT-H_8-G_mean	96.28%	52.94%	3.70%	6.92%	52.51%
— GAT-H_8-G_max	96.65%	85.71%	12.35%	21.58%	59.94%
— GAT-H_8-G_sum	96.89%	84.75%	20.58%	33.11%	65.76%

Table 9: Comprehensive Performance Matrix: Frequency Baseline vs. GNN Architectures

A: The simple GCN variants show strong performance on the majority malware class but relatively low capability on benign samples. The `Amax` variant experiences a complete majority class collapse (0.00% benign recall). While `Amean` and `Asum` aggregators improve slightly, benign recall remains constrained at 3.70% and 8.64% respectively, with benign F1-scores capping at 15.11%.

GraphSAGE models perform best among the GNNs, achieving the highest overall test accuracies—peaking at 97.54% for `SAGE-L_max-G_mean`—and substantially improving the minority class trade-off. The `SAGE-L_max-G_sum` configuration achieves an all-time high benign recall of 53.09% and a top benign F1-score of 60.71%. Conversely, the `SAGE-L_mean-G_max` variant prioritizes confidence, securing a peak benign precision of 86.67% at the expense of a lower recall (16.05%).

The GAT architectures deliver a modest improvement over GCNs, maintaining overall test accuracy between 96.26% and 96.99%, but generally lag behind GraphSAGE. The top variant, `GAT-H_4-G_sum`, reaches a high benign precision of 85.07%, but its benign recall remains limited at 23.46% (36.77% F1-score). Other combinations, such as `GAT-H_1-G_max`, undergo a total majority class prediction collapse (0.00% recall).

Compared to the simple frequency baseline (97.36% accuracy, 50.86% benign F1, and 36.63% benign recall), the GNN families demonstrate mixed results. All GCN models and weaker GAT variants fail to match the baseline benchmark. However, the top-performing GraphSAGE and GAT models comfortably exceed it, with `SAGE-L_max-G_sum` achieving an absolute peak Macro F1-score of 79.69% and pushing benign recall up to 53.09%.

In comparison with previous non-graph pipelines, the simpler GCN and GAT variants perform comparably to or slightly worse than raw sequential identifiers and vanilla RNNs, which similarly suffer from severe under-detection. On the other hand, the best-performing GraphSAGE configurations match or exceed the advanced learnable embedding FFNNs and gated sequential models, outperforming the highest sequential benchmark set by the Unidirectional LSTM (55.21% benign F1).